

Typing Behavior and Scope Resolution in Python 3

CS 660: Advanced Programming Language Theory

Aaron Foster

Prof. Shadha Tabatabai

Southern New Hampshire University

July 26, 2026

Files Submitted

File	Purpose
<code>typing_mismatch.py</code>	Part One, steps 2-3 and 5. Five deliberate type mismatches plus a compile-stage demonstration.
<code>typing_corrected.py</code>	Part One, step 4. The repaired counterpart of every mismatch.
<code>lexical.py</code>	Part Two, steps 7 and 9. Provided starter code, annotated with resolution traces; Examples 3b and 5 added.
<code>dynamic.py</code>	Part Two, steps 8-10. Dynamic scoping simulated with an explicit environment stack.
<code>program_output.txt</code>	Captured console output of all four scripts.

All four scripts run under CPython 3.13 with no third-party dependencies.

Part One: Typing Behavior in Code

1. Identifying the Type System

The provided language is Python 3, executed here on CPython 3.13. Python's type system is defined by four properties, and every experiment in Part One follows from one of them.

Dynamically typed. A variable is never declared with a type. A name is only a reference, and the type belongs to the object the name is bound to rather than to the name itself; binding a name is an operation that happens while the program runs (Python Software Foundation, 2026a, §4.2.1). The same name may therefore reference an integer on one line and a string on the next, and no stage of translation objects to that.

Strongly typed. Python does not silently coerce values across unrelated types to make an operation succeed. The expression `"5" + 5` raises rather than guessing whether the programmer meant 10 or `"55"`. Dynamic and weak are independent axes: Python is dynamic but strict about what operations mean.

Duck typed. Operations dispatch on the behavior an object supports, not on a declared class. An operand is acceptable if its type implements the corresponding special method, which is why some mismatches produce a wrong answer instead of an error (Mismatch 3 below).

Gradually typed. Since PEP 484, annotations such as `def apply_discount(price: float) -> float` may be written, but the interpreter treats them purely as metadata. They are stored on the function object and never consulted during execution (van Rossum et al., 2014). The annotation is a claim, not a guarantee, and Mismatch 2 exploits exactly that gap.

2. Modifying the Code to Demonstrate Type Mismatches

The file `typing_mismatch.py` introduces five violations. Each is syntactically legal, so the module compiles without complaint. A small harness executes each one and reports what the interpreter did, which allows the program to run to completion and display the full set of responses rather than halting on the first failure.

Mismatch 1 — implicit coercion refused

```
def build_receipt_line(item, quantity):  
    return item + " x" + quantity          # str + int  
  
build_receipt_line("widget", 3)
```

Mismatch 2 — an annotation that is not enforced

```
def apply_discount(price: float, percent: float) -> float:
```

```
    return price * (1 - percent)
```

```
apply_discount("19.99", 0.10)          # str passed where float is declared
```

Mismatch 3 — a mismatch that never raises

```
def repeat_label(label: str, times: int) -> str:
    return label * times
```

```
repeat_label("7", 3)                    # caller wanted 21
```

Mismatch 4 — a value that supports neither operation used on it

```
def first_id(records: List[int]) -> int:
    print(len(records))
    return records[0]
```

```
first_id(42)                            # int is neither sized nor subscriptable
```

Mismatch 5 — a guaranteed error that is never discovered

```
def unreachable_mismatch():
    total = 42
    return "Order total: " + total      # certain TypeError
```

```
# ...and the function is never called.
```

3. Compiler and Interpreter Behavior

CPython translates source in stages: it tokenizes the text, parses it into an abstract syntax tree, compiles that tree to bytecode, and only then executes the bytecode in the evaluation loop. Type checking appears in none of the first three stages. Disassembling a trivial addition shows why:

```
def f(a, b): return a + b
```

RESUME	0
LOAD_FAST_LOAD_FAST	1 (a, b)
BINARY_OP	0 (+)

RETURN_VALUE

The emitted `BINARY_OP` instruction records the operator and nothing about the operands. The compiler cannot know what `a` and `b` will hold, so it defers the entire question. At runtime the interpreter asks the left operand's type to perform the operation, then the right operand's type to perform the reflected version. When both decline, the interpreter raises `TypeError` and reports the two concrete types involved. The messages below are verbatim program output.

Mismatch	Interpreter response	What it reveals
1. <code>str + int</code>	<code>TypeError: can only concatenate str (not "int") to str</code>	Strong typing. The interpreter names both types and refuses to guess an intent.
2. annotated float given <code>str</code>	<code>TypeError: can't multiply sequence by non-int of type 'float'</code>	The annotation was never checked. The error surfaces from the multiplication operator, not from the parameter, so the message describes a symptom several steps removed from the cause.
3. <code>str * int</code>	Returned <code>'777' (type: str)</code>	No error at all. Duck typing made the operation meaningful for these operand types, so a type-related defect became a silent wrong answer.
4. <code>int</code> used as a list	<code>TypeError: object of type 'int' has no len()</code>	The failure occurs on the first line that assumes a sequence protocol, after the function has already been entered.
5. never executed	No error	Unexecuted code is unchecked code. The defect ships.

Table 1. Interpreter behavior produced by each mismatch (see `program_output.txt`).

Two results deserve emphasis. Mismatch 3 shows that a strong type system is not a complete type system: `str * int` is a legitimate, well-defined operation, so no rule was broken even though the programmer's intent was. Mismatch 5 shows the boundary of dynamic checking. The module imports, compiles, and runs cleanly while containing a certain `TypeError`, because detection requires execution and that line never executes.

4. Correcting the Type Mismatches

The file `typing_corrected.py` repairs each violation. Because the language supplies no declaration to lean on, the corrections follow three rules that dynamic typing actually makes available.

- Convert explicitly. Python will not coerce, so the conversion must be stated: `f-strings`, `str()`, `float()`, `int()`.

- Validate at the boundary. A function must reject values it cannot handle and must fail with a message that names the offending argument, since no earlier stage will do it.
- Annotate for the reader and for the checker. Hints do not execute, but they document intent and allow an external static analyzer to find mismatches before execution.

#	Before	After
1	<code>return item + " x" + quantity</code>	<code>return f"{item} x{quantity}"</code>
2	<code>return price * (1 - percent)</code>	<code>price = _coerce_number(price, "price")</code> <code>return round(price * (1 - percent), 2)</code>
3	<code>return label * times</code>	<code>if not isinstance(times, int) or</code> <code>isinstance(times, bool):</code> <code>raise TypeError(...)</code> <code># plus add_quantities() for the</code> <code>arithmetic actually intended</code>
4	<code>return records[0]</code>	<code>if not isinstance(records, Sequence):</code> <code>raise TypeError(...)</code> <code>if not records: raise ValueError(...)</code> <code>return records[0]</code>
5	<code>return "Order total: " + total</code>	<code>return f"Order total: \$</code> <code>{_coerce_number(total, 'total'):.2f}"</code> <code># and the function is now actually</code> <code>called</code>

Table 2. Each mismatch and its correction.

One detail in the guard for Mismatch 3 is worth noting: `bool` is a subclass of `int` in Python, so a plain `isinstance(times, int)` check silently accepts `True`. The corrected code excludes it explicitly. This is a good illustration of why runtime validation in a dynamic language must be written with the language's actual object model in mind rather than an idealized one.

The corrections did not eliminate errors; they relocated them. Invalid input now fails at the function boundary with the argument named, instead of failing several operations later with an operator-level message. Mismatch 5 required a second kind of fix, because correcting the expression alone would have left the function untested and therefore still unchecked.

5. Typing at Runtime Versus Compile Time

The demonstration in `show_compile_stage()` separates the two stages directly. A source string containing a certain type error compiles successfully, while a source string containing a syntax error is rejected before anything runs.

Stage	What is verified	Evidence from this activity	Outcome
Compile — grammar	Tokens, structure, indentation	<code>compile('x == 42')</code>	<code>SyntaxError</code> , raised before execution
Compile — types	Nothing	<code>compile('x = "total: " + 42')</code>	Compiles successfully
Compile — whole module	Nothing about types	<code>py_compile</code> of <code>typing_mismatch.py</code>	Bytecode written despite five mismatches
Runtime — line executed	Operand types, at operator dispatch	<code>build_receipt_line("width", 3)</code>	<code>TypeError</code> , at that line
Runtime — line not executed	Nothing	<code>unreachable_mismatch()</code>	Never detected

Table 3. What Python verifies, and when.

The contrast with a statically typed language is sharp. In Java or C#, every one of these mismatches except the third would be a compile error, reported for every line of the file in a single pass, whether or not that line ever runs. Coverage is total and independent of test quality. Python trades that guarantee away and gets it back only partially, through two mechanisms:

- Runtime checking, which is precise about the actual values involved but only ever examines the paths a given execution takes. Its coverage equals the coverage of the test suite.
- Optional static analysis, through an external checker such as `mypy` or `pyright` reading the PEP 484 annotations. This restores compile-time detection for annotated code, including Mismatch 5, but it is opt-in tooling rather than a property of the language, and it is only as complete as the annotations are.

The practical consequence is that in Python, type confidence is produced by test coverage and tooling discipline, whereas in a statically typed language it is produced by the compiler.

6. Effects on Clarity, Flexibility, and Error Prevention

Clarity

Dynamic typing cuts both ways. Signatures are short and uncluttered, and the absence of declarations removes a whole category of ceremony. But a signature also stops carrying a contract. Reading `def apply_discount(price, percent)` tells a maintainer nothing about whether a string is acceptable, so the answer must be recovered from the body, from the call sites, or from the docstring. Annotations restore that information at the cost of the brevity that motivated the design, which is why they are optional. Clarity in Python is a matter of author discipline rather than a language guarantee.

Flexibility

This is where the model pays. Duck typing lets one function accept any object supporting the required behavior, so generic code needs no interface hierarchy, no generics, and no explicit implementation declarations. Objects can be composed and extended at runtime. The same property drives Python's strength in scripting, prototyping, data analysis, and testing, where a mock only needs to respond to the calls that are actually made. The cost is that the set of acceptable inputs is never written down anywhere the machine can check, which is precisely what Mismatch 2 exploited.

Error Prevention

Prevention is the weakest of the three. The type system prevents nonsensical operations from proceeding, and it does so with informative messages, but only on the lines that actually run. Mismatch 5 demonstrates the boundary directly: a certain defect can be shipped in a module that compiles and passes every test it happens to exercise. Mismatch 3 demonstrates a second boundary, in which a real defect produces a plausible value instead of an exception and is therefore not prevented at all. The practical response is the one implemented in `typing_corrected.py`: validate at boundaries, annotate consistently, and treat coverage as a type-safety mechanism rather than merely a correctness one.

Part Two: Scope Resolution in Nested Functions

7. Demonstrating Lexical Scoping

Python resolves an unqualified name by the LEGB rule: Local, then Enclosing, then Global, then Built-in (Python Software Foundation, 2026a, §4.2.2). The decisive step is the second. Enclosing means the locals of functions that this function is *written inside*. That chain is fixed when the source is compiled and does not depend on who calls the function. This is lexical, or static, scoping.

Running the provided `lexical.py` produces:

Example 1:

Inner sees: outer

Outer has: outer

Example 3:

1

2

2

Example 2:

Inner sees: 20

Middle has: 20

Outer has: 10

Example 4:

Inner: inner

Outer: outer

Global: global

- Example 1 confirms the basic closure: `inner()` does not bind `x`, so resolution steps out one lexical level and finds the enclosing binding.
- Example 2 confirms that the nearest enclosing binding wins. `inner()` reports 20 from `middle`, not 10 from `example_2`, and the search stops at the first hit. Because `middle` never declared `nonlocal`, its `x = 20` created a new name that shadowed the outer one; the final line printing 10 proves the outer binding was untouched.
- Example 3 confirms that a closure holds a live binding rather than a copy. `increment()` declares `nonlocal count`, so both calls mutate the same binding, and the final print of 2 matches the counter. Without `nonlocal`, `count += 1` would read a name it is also assigning and raise `UnboundLocalError`.
- Example 4 confirms shadowing across three levels. Each function assigns `name`, so each satisfies the Local step immediately and no enclosing scope is consulted. Three distinct bindings coexist and no assignment leaks outward. Note that the outermost binding is labeled "global" but is in fact a local of `example_4`; the true module global appears only in the added Example 5.

Examples 1 through 4 share a structural feature that turns out to matter: in each one, a function is called from inside the same function it is written in. The lexical chain and the call chain are the same chain. Two examples were therefore added to separate them, because a comparison built only on Examples 1 through 4 would show no difference at all.

```
status = "module-level"          # the real module global

def report():
    print(status)                 # written inside nothing -> Global

def audit_run():
    status = "auditing"          # a local of audit_run
    report()                     # prints: module-level

def deploy_run():
    status = "deploying"         # a local of deploy_run
    report()                     # prints: module-level
```

Example 5 gives the same answer from both callers. Example 3b returns the counter closure out of example_3 and calls it after that call has finished; it prints 3, because the binding travels with the function object and does not expire when the call does.

8. Simulating Dynamic Scoping

Python provides no dynamic scoping, so `dynamic.py` replaces LEGB resolution with an explicit environment stack. Every call pushes a frame holding the names it binds; reading a free name walks the stack from the most recent call outward and takes the first frame that holds it; returning pops the frame.

```
_dyn_stack = []                  # a stack of frames, innermost call last

@contextlib.contextmanager
def dyn_frame(**bindings):
    _dyn_stack.append(dict(bindings)) # PUSH on call

    try:
        yield

    finally:
        _dyn_stack.pop()            # POP on return -- the binding expires
```

```

def dyn_get(name):

    for frame in reversed(_dyn_stack): # newest CALLER outward

        if name in frame:

            return frame[name]

    raise NameError(f"{name!r} is not bound in the current call stack")


def dyn_set(name, value):                # the analogue of nonlocal

    for frame in reversed(_dyn_stack):

        if name in frame:

            frame[name] = value

    return

```

The difference from `lexical.py` is a single word. Lexical resolution walks the chain of functions a name is written inside; dynamic resolution walks the chain of functions that are currently running. The `finally` clause is what makes the simulation faithful: a dynamic binding lives exactly as long as the call that created it, which is the property that breaks closures in Example 3b.

9. Tracing Variable Resolution

Both files carry in-line comments marking every binding and every lookup. Comments beginning with BIND record where a name is bound; comments beginning with TRACE record how a free name is resolved, naming the scope or frame that satisfies it and the candidates that were skipped. The two traces below are transcribed from those comments.

Example 2, where the models agree

Step	Action	Lexical result	Dynamic result
1	example_2 binds x = 10	example_2.x = 10	stack: [{x:10}]
2	middle binds x = 20 (no nonlocal, so this shadows)	middle.x = 20	stack: [{x:10}, {x:20}]
3	inner reads x	not Local; enclosing chain inner → middle hits 20; the 10 is never reached	not in top frame; next frame down holds 20; the 10 is never reached

Step	Action	Lexical result	Dynamic result
4	middle reads x	middle.x = 20	top frame = 20
5	middle returns, example_2 reads x	example_2.x = 10, untouched	frame popped, uncovering 10

Table 4. Example 2 resolves identically under both models — because inner is written inside middle and also called by middle.

Example 5, where the models diverge

Step	Action	Lexical result	Dynamic result
1	module binds status = "module-level"	the true Global	stack: [{status:'module-level'}]
2	audit_run binds status = "auditing"	a local of audit_run	pushed on the stack
3	audit_run calls report(); report reads status	report is nested inside nothing, so there is no Enclosing step; resolution goes to Global → 'module-level'	newest frame holds it → 'auditing'
4	deploy_run binds status = "deploying", then calls report()	'module-level' again	'deploying'
5	report() called with nothing bound	'module-level'	NameError — nothing on the stack

Table 5. The same function, the same source line, two different answers.

The mechanism is visible in the trace comment on `report()`. Lexically, the function is written at module level and is nested inside nothing, so there is no enclosing scope to search and resolution goes straight to the global binding on every call, regardless of the caller. Dynamically, the same lookup takes whatever the caller most recently bound, so the function's behavior is decided at the call site.

10. Comparing the Outcomes

Running `dynamic.py` against `lexical.py` produces this comparison, printed by the program itself:

Case	Lexical	Dynamic	Verdict
Ex 1 — inner sees x	outer	outer	same
Ex 2 — inner sees x	20	20	same
Ex 3 — counter results	1, 2, 2	1, 2, 2	same

Case	Lexical	Dynamic	Verdict
Ex 3b — counter called after its defining call returned	3	NameError	DIFFERS
Ex 4 — inner sees name	inner	inner	same
Ex 5 — report() from audit_run	module-level	auditing	DIFFERS
Ex 5 — report() from deploy_run	module-level	deploying	DIFFERS
Ex 5 — report() with nothing bound	module-level	NameError	DIFFERS

Table 6. Divergence summary, from the output of dynamic.py.

The pattern in the verdict column is the finding. Examples 1 through 4 agree, and they agree for a reason rather than by luck: in each, the callee was written inside the function that called it, so the lexical chain and the call chain coincide. The models split only once those chains differ. In everyday nested-function code the two rules are indistinguishable, which is exactly what makes dynamic scoping dangerous in the cases where they are not.

Predictability

Lexical scoping is predictable because the answer is a property of the text. Every free name in a function can be resolved by reading outward from the function to the enclosing source, and the result is the same on every call, from every caller, on every thread. The compiler can perform the resolution, which is why it can also report an unbound name as an error and why editors can implement jump-to-definition at all.

Dynamic scoping makes the answer a property of the run. In the trace above, `report()` has three different behaviors from three different callers without a single character changing in its body, and one of them is a crash. The set of possible results is the set of possible call paths, so it cannot be enumerated by reading the function, and in a concurrent program it cannot be enumerated at all. Example 3b sharpens this: a function that worked correctly became a `NameError` simply because it outlived its caller, and then — worse — returned `101` when called inside an unrelated frame that happened to use the same name. Silent capture by an unrelated caller is the characteristic failure of the model, and it produces a wrong answer rather than an error.

Readability

Under lexical scoping, the code needed to understand a function is bounded and locatable: the function plus the scopes it is nested in. Review, testing, and reasoning are all local activities, and a function can be tested by calling it directly because calling it directly is the same as calling it from anywhere else.

Under dynamic scoping, understanding a function requires knowing every path that reaches it, so the reading unit grows from one function to the whole call graph. Two costs follow. First, the names a function depends on become an invisible part of its interface: renaming a local variable

in one function can change the behavior of another function written by someone else, and nothing in either file records the coupling. Second, a function can no longer be tested in isolation, because its result depends on the frames above it. There is a modest readability gain — signatures stay short because contextual values need not be threaded through every intermediate call — but it is purchased with a dependency that no reader can see.

11. Which Scoping Model Is Preferable

Lexical scoping should be the default, and essentially every language designed since the 1970s has settled there. The reason is visible in Table 6: the two models agree on ordinary nested code and disagree exactly where a reader would be surprised. For any codebase maintained by more than one person, resolution that can be determined by reading the source is worth more than the convenience dynamic scoping offers. Lexical scoping is also a precondition for closures, higher-order functions, and reliable static analysis, and it is the only model that behaves sanely under concurrency, since a lexical binding does not depend on which stack a call happens to be running on.

Dynamic scoping remains preferable in one identifiable situation: a value that is genuinely contextual, is consumed far from where it is set, and would otherwise have to be threaded through every intermediate function that does not care about it. Typical cases include a request or correlation ID in a server, the current locale or output stream, a database transaction or connection, and temporary overrides in tests. Lexically, each of these forces the value into the signature of every function on the path, which couples unrelated code to a parameter it merely forwards. Dynamically, the caller binds it once and every frame below sees it. This is why Common Lisp keeps special variables, why Emacs Lisp uses dynamic binding for user-facing configuration, and why Perl provides `local`.

Context	Preferred model	Rationale
Application and library code, any team size	Lexical	Resolution is determinable from the source; review and static analysis stay local.
Closures, callbacks, higher-order functions	Lexical	Example 3b: dynamic bindings expire with the call, so a returned function breaks.
Concurrent or asynchronous code	Lexical	Dynamic bindings are a property of a call stack, and there is more than one.
Cross-cutting context: request ID, locale, transaction	Dynamic, scoped explicitly	Avoids threading a value through functions that only forward it.
Test fixtures and temporary overrides	Dynamic, scoped explicitly	Rebind for the duration of a call, restore automatically on exit.

Context	Preferred model	Rationale
Interactive and user-extensible systems	Dynamic	Users rebind behavior at runtime without editing the definitions.

Table 7. Recommended model by context.

Python's own resolution of this tension is instructive, and it is the recommendation this activity supports. The language keeps lexical scoping for all ordinary name resolution and provides the dynamic behavior separately, as an explicit, opt-in facility: `contextvars.ContextVar` holds a value that is set by a caller and read by any frame below it, with correct behavior across threads and asynchronous tasks (Python Software Foundation, 2026b). Crucially, a context variable is declared, is named at the point of use, and cannot be captured by accident — which removes the silent-capture failure of Example 5 while keeping the convenience of Table 7's last three rows. The general principle is that dynamic scoping is a useful mechanism and a poor default: it should be requested explicitly for the values that genuinely need it, never applied to every variable in the language.

Program Output

Insert a screenshot of your own console session below. Run each script in order from the Module Three Activity directory:

```
python3 typing_mismatch.py
```

```
python3 typing_corrected.py
```

```
python3 lexical.py
```

```
python3 dynamic.py
```

```
~/Desktop/COMP-SCI/MASTERS/CS 660/MOD 3/Module Three Activity 21:30:14
● > python3 typing_mismatch.py
=====
PART ONE - TYPE MISMATCHES UNDER PYTHON'S DYNAMIC TYPE SYSTEM
=====

[R] RUNTIME STAGE

1. str + int -> build_receipt_line('widget', 3)
   -> RAISED TypeError: can only concatenate str (not "int") to str

2. annotation ignored -> apply_discount('19.99', 0.10)
   -> RAISED TypeError: can't multiply sequence by non-int of type 'float'

3. silent mismatch -> repeat_label('7', 3) [expected 21]
   -> RETURNED '777' (type: str)

4. int used as list -> first_id(42)
   -> RAISED TypeError: object of type 'int' has no len()

5. unreachable_mismatch() is defined but never called
   -> NO ERROR. The guaranteed TypeError is never discovered.

[C] COMPILE STAGE
compile('x = "total: " + 42') -> SUCCESS (no type check)
compile('x == 42') -> SyntaxError: invalid syntax
py_compile of this whole module -> SUCCESS (bytecode written)
CONCLUSION: the compile stage validates GRAMMAR, not TYPES.

=====
Module reached the end. Four mismatches raised at runtime, one was
never detected, and zero were caught before execution began.
=====
```

```
• > python3 typing_corrected.py
```

```
=====
PART ONE - CORRECTED CODE, ALIGNED WITH PYTHON'S TYPING RULES
=====
```

[A] The same calls that failed before now succeed

1. build_receipt_line('widget', 3)
→ RETURNED 'widget x3' (type: str)
2. apply_discount('19.99', 0.10)
→ RETURNED 17.99 (type: float)
3. add_quantities('7', 3) [the intended arithmetic]
→ RETURNED 10.0 (type: float)
4. first_id([101, 102, 103])
→ RETURNED 101 (type: int)
5. format_total(42) [formerly the unreachable mismatch]
→ RETURNED 'Order total: \$42.00' (type: str)

[B] Genuinely bad input now fails loudly, at the boundary, with a message that names the offending argument

6. apply_discount('nineteen', 0.10)
→ RAISED TypeError: price must be numeric, got non-numeric str: 'nineteen'
7. repeat_label('7', '3')
→ RAISED TypeError: times must be int, got str
8. first_id(42)
→ RAISED TypeError: records must be a sequence of int, got int
9. first_id([])
→ RAISED ValueError: records is empty; there is no first id

```
=====
Errors did not disappear -- they moved. They now surface at the
function boundary with the argument named, instead of deep inside
an expression with an operator-level message.
=====
```

```
• > python3 lexical.py
```

Example 1:

Inner sees: outer
Outer has: outer

Example 2:

Inner sees: 20
Middle has: 20
Outer has: 10

Example 3:

1
2
2

Example 3b: (added) the closure after its defining call returned
example_3 has already returned; calling my_counter() anyway:
→ 3

Example 4:

Inner: inner
Outer: outer
Global: global

Example 5: (added) definition site vs. call site

```
audit_run (its own local status = 'auditing')
report() sees status = 'module-level'
deploy_run (its own local status = 'deploying')
report() sees status = 'module-level'
module-level status is still 'module-level'
```

```
~/Desktop/COMP-SCI/MASTERS/CS 660/MOD 3/Module Three Activity 21:52:20
> python3 dynamic.py
Example 1:
Inner sees: outer
Outer has: outer

Example 2:
Inner sees: 20
Middle has: 20
Outer has: 10

Example 3:
1
2
2

Example 3b: (added) the counter after its frame is gone
stack after example_3 returned: (empty)
my_counter() -> NameError: 'count' is not bound anywhere in the current call stack (depth 1)
called inside a frame with count=100 -> 101

Example 4:
Inner: inner
Outer: outer
Global: global

Example 5: (added) definition site vs. call site
audit_run (binds status = 'auditing')
report() sees status = 'auditing'
deploy_run (binds status = 'deploying')
report() sees status = 'deploying'
outermost status is still 'module-level'
calling report() with an empty stack:
-> NameError: 'status' is not bound anywhere in the current call stack (depth 1)

=====
SUMMARY OF DIVERGENCE
=====
case                lexical      dynamic      verdict
-----
Ex 1 inner sees x    outer       outer       same
Ex 2 inner sees x    20          20          same
Ex 3 counter results 1, 2, 2     1, 2, 2     same
Ex 3b counter after return 3 (works)   NameError   DIFFERS
Ex 4 inner sees name inner        inner       same
Ex 5 report() in audit module-level  auditing   DIFFERS
Ex 5 report() in deploy module-level  deploying   DIFFERS
Ex 5 report() bare call module-level  NameError   DIFFERS
=====

Examples 1-4 agree because each callee was written inside the same
function that called it -- the lexical chain and the call chain were
the same chain. The models split only once those chains differ.
=====
```

Captured Output for Reference

The full console transcript is also submitted as `program_output.txt`. The key excerpts are reproduced below.

typing_mismatch.py

```
[R] RUNTIME STAGE

1. str + int -> build_receipt_line('widget', 3)
```

-> RAISED TypeError: can only concatenate str (not "int") to str

2. annotation ignored -> apply_discount('19.99', 0.10)

-> RAISED TypeError: can't multiply sequence by non-int of type 'float'

3. silent mismatch -> repeat_label('7', 3) [expected 21]

-> RETURNED '777' (type: str)

4. int used as list -> first_id(42)

-> RAISED TypeError: object of type 'int' has no len()

5. unreachable_mismatch() is defined but never called

-> NO ERROR. The guaranteed TypeError is never discovered.

[C] COMPILE STAGE

compile('x = "total: " + 42') -> SUCCESS (no type check)

compile('x == 42') -> SyntaxError: invalid syntax

py_compile of this whole module -> SUCCESS (bytecode written)

CONCLUSION: the compile stage validates GRAMMAR, not TYPES.

typing_corrected.py

[A] The same calls that failed before now succeed

1. build_receipt_line('widget', 3) -> 'widget x3'

2. apply_discount('19.99', 0.10) -> 17.99

3. add_quantities('7', 3) -> 10.0

4. first_id([101, 102, 103]) -> 101

5. format_total(42) -> 'Order total: \$42.00'

[B] Genuinely bad input now fails loudly, at the boundary

6. apply_discount('nineteen', 0.10)

-> RAISED TypeError: price must be numeric, got non-numeric str: 'nineteen'

7. repeat_label('7', '3')

-> RAISED TypeError: times must be int, got str

8. first_id(42)

-> RAISED TypeError: records must be a sequence of int, got int

9. first_id([])

-> RAISED ValueError: records is empty; there is no first id

lexical.py — Examples 3b and 5

Example 3b: (added) the closure after its defining call returned

example_3 has already returned; calling my_counter() anyway:

-> 3

Example 5: (added) definition site vs. call site

audit_run (its own local status = 'auditing')

report() sees status = 'module-level'

deploy_run (its own local status = 'deploying')

report() sees status = 'module-level'

module-level status is still 'module-level'

dynamic.py — Examples 3b and 5

Example 3b: (added) the counter after its frame is gone

stack after example_3 returned: (empty)

my_counter() -> NameError: 'count' is not bound anywhere in the

current call stack (depth 1)

called inside a frame with count=100 -> 101

Example 5: (added) definition site vs. call site

audit_run (binds status = 'auditing')

report() sees status = 'auditing'

deploy_run (binds status = 'deploying')

report() sees status = 'deploying'

outermost status is still 'module-level'

calling report() with an empty stack:

-> NameError: 'status' is not bound anywhere in the current call stack

References

Python Software Foundation. (2026a). The Python language reference (Version 3.13), §4.2: Naming and binding. Retrieved July 26, 2026, from <https://docs.python.org/3/reference/executionmodel.html>

Python Software Foundation. (2026b). contextvars — Context variables (Python 3.13 standard library documentation). Retrieved July 26, 2026, from <https://docs.python.org/3/library/contextvars.html>

Python Software Foundation. (2026c). Built-in types (Python 3.13 standard library documentation). Retrieved July 26, 2026, from <https://docs.python.org/3/library/stdtypes.html>

van Rossum, G., Lehtosalo, J., & Langa, Ł. (2014). PEP 484 – Type hints. Python Software Foundation. <https://peps.python.org/pep-0484/>